

FDE Echo 实习生笔试

企业岗位经验 Skill 生成平台设计

把优秀员工的岗位经验，通过自然语言描述，
自动转换为可执行、可评测、可版本管理的 AI 员工能力资产。

姓名	袁陈博
学校	雷丁大学 亨利商学院
专业	数字商业与数据分析

在线 Demo <https://sparkdata.onrender.com>

代码仓库 <https://github.com/chenboyuan0818/sparkdata>

在线 Demo 无需任何配置即可直接体验完整流程。
免费实例在无访问时会休眠，首次打开约需一分钟唤醒。

目录

一、产品设计文档（PRD）

二、技术方案说明

三、面试问题回答

可运行 Demo 见封面链接，代码与自动化测试均在仓库中。

企业岗位经验 Skill 生成平台 — 产品设计文档 (PRD)

版本 v1.0 | 面向数花智算 DataAgent 实习生面试题

关联文档: 《02_技术方案说明》《03_面试问题回答》

一、一句话定义

把优秀员工"怎么看数据、看哪些指标、异常了怎么归因、然后该做什么"的岗位经验, 通过自然语言描述, 自动沉淀为可复用、可版本管理、可评测的 AI 员工能力资产 (Skill)。

二、问题定义

2.1 我们观察到的真实现象

企业里最值钱的分析能力, 不在报表系统里, 在人脑子里。

一个做了三年的抖音直播运营, 看到"昨天 GMV 掉了 30%", 他的动作是自动的:

1. 先看是**流量问题**还是**转化问题**——拉曝光量、进入率、停留时长
2. 如果是流量掉了, 看是**自然流量**还是**付费流量**——分渠道拆
3. 如果是转化掉了, 看是**商品问题**还是**话术问题**——拆 SKU 点击转化率、看讲解时长分布
4. 定位到具体原因后, 他知道**对应的动作是什么**——是调价、换品、改脚本, 还是加投流

这四步在他脑子里可能只用 30 秒。但是:

- 这套东西从没被写下来过。他自己都说不清"我就是这么看的"
- 新人复制不了。新人拿到同样的数据, 只会说"GMV 掉了 30%", 说不出为什么、也说不出怎么办
- 他离职了, 能力归零。团队要重新养一个人, 成本 6-12 个月
- 他也不可能被复制成 10 个。公司有 20 个直播间, 他只能盯 3 个

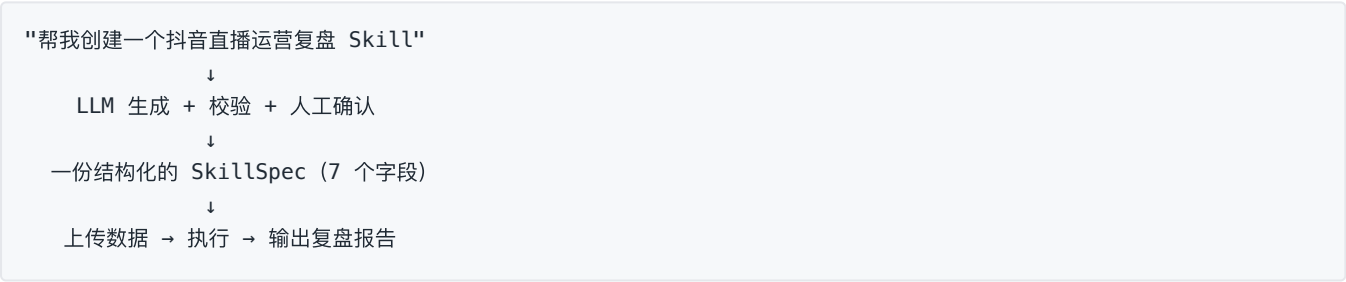
2.2 现有方案为什么不够

方案	为什么不够
写 SOP 文档	文档是死的，不会执行。人还是要一步步照着做，没有节省时间
让业务同学自己用 ChatGPT	每个人 Prompt 水平不同、口径不同、结果不可复现；今天问和明天问答案不一样；老板不敢用这个结果做决策
BI 报表/数据看板	只回答"发生了什么"，不回答"为什么"和"接下来怎么办"。看板做得再多，还是需要一个懂业务的人来解读
找外包开发定制智能体	一个 Skill 排期两周，20 个岗位场景做不完；业务口径一变就要返工；业务方永远等技术方

核心矛盾：懂业务的人不会写代码，会写代码的人不懂业务。

2.3 我们的解法

让懂业务的人用自然语言把经验说出来，系统自动结构化成一份可执行的资产。



三、目标用户

3.1 三类角色（Skill 的"生产—消费—治理"闭环）

角色	典型画像	核心诉求	在平台上做什么
Skill 创建者 (业务专家 / 主管)	电商运营负责人、销售 Leader、增长负责人。有 3 年以上经验，但不会写代码	"我的经验能不能被复制，而不是我天天救火"	用自然语言创建 Skill → 审核系统生成的内容 → 校准指标口径和阈值 → 发布
Skill 使用者 (一线执行 / 新人)	运营专员、销售代表、门店店长	"我不知道该看哪些数据，也不知道数据异常了该做什么"	选 Skill → 上传数据 → 拿到分析报告和行动建议
Skill 管理者 (数据/AI 平台负责人)	企业数字化负责人、AI 平台 owner	"我要保证公司里跑的 AI 分析是靠谱的、可管的、可审计的"	管理 Skill 资产目录、审批发布、看评测分数、控权限、看使用数据

3.2 首批切入场景（优先级排序）

按"经验密度高 + 数据结构化程度高 + 结果可验证"三个维度筛选：

优先级	场景	为什么优先
P0	电商/直播运营复盘	数据标准化程度最高（平台后台可导出）、指标体系成熟、复盘频次高（日/周）、效果可验证
P0	销售经营分析	CRM 数据结构清晰、漏斗模型标准、管理层刚需
P1	用户增长分析	数据来源多样（埋点+渠道），归因逻辑复杂，价值高但难度大
P1	零售门店分析	门店多、复制需求最强，但数据质量参差

四、核心概念：Skill 是什么

4.1 Skill vs Prompt：这是本平台最关键的定义

Prompt 是一次性的指令，Skill 是被结构化封装的资产。

维度	Prompt	Skill
形态	一段自然语言文本	一份结构化 Schema（JSON）
状态	无状态，用完即弃	有版本、有归属人、有生命周期
输入约定	无契约，喂什么算什么	有明确的 <code>input_schema</code> ，字段缺失会拦截
计算方式	全部交给模型，包括算数	数值走确定性代码，模型只做解读
输出	每次格式都可能不同	固定 <code>output_template</code> ，结构稳定可下游消费
质量保障	靠人肉试	有评测集、有回归测试、有评分卡
复用	靠复制粘贴，容易走样	平台统一分发，一处修改全员生效
可管理性	散落在个人聊天记录里	集中在资产库，可授权、可审计、可下架

一个类比： Prompt 是"你口头交代同事一句话"， Skill 是"你把这件事写成了带输入输出规范、带检查清单、带验收标准的岗位作业指导书，并且这份指导书本身可以自动执行"。

4.2 Skill 的七个核心组成（对应题目要求）

#	字段	作用	举例（抖音直播复盘 Skill）
1	Skill 名称 name	资产标识，用于检索和调用	抖音直播运营复盘分析
2	Skill 描述 description	让使用者判断"这个是不是我要的"	基于单场直播的流量、互动、转化、商品四维数据，定位 GMV 波动原因并输出可执行改进动作
3	使用场景 use_cases	明确边界，防止误用	① 单场直播结束后复盘 ② 周维度多场对比 ③ GMV 异常波动归因
4	输入数据定义 input_schema	数据契约——决定 Skill 能不能真的跑起来	场次ID(string,必填) 曝光人数(int,必填) 进入直播间人数(int,必填) 平均停留时长(float,秒) 商品点击人数(int) 成交订单数(int) GMV(float,元) ...
5	分析流程 analysis_flow	Skill 的骨架——把专家的思考路径固化下来	Step1【计算】算出进入率/停留/点击率/转化率四个核心指标 Step2【计算】与基准期对比，算变化率 Step3【判断】按漏斗顺序定位第一个显著下滑的环节 Step4【LLM】对该环节做归因分析 Step5【LLM】匹配行动建议库，输出改进动作
6	Agent Prompt agent_prompt	角色设定 + 分析准则 + 禁止事项	"你是资深抖音直播运营专家..... 【禁止】不得自行计算任何数值，所有数字必须引用上游计算结果；【禁止】不得在数据不足时臆测原因，应明确指出缺失字段"
7	输出结果模板 output_template	保证输出结构稳定，可被下游消费	一、本场核心指标概览（表格） 二、漏斗诊断结论 三、根因分析 四、改进建议（按优先级） 五、数据说明与局限

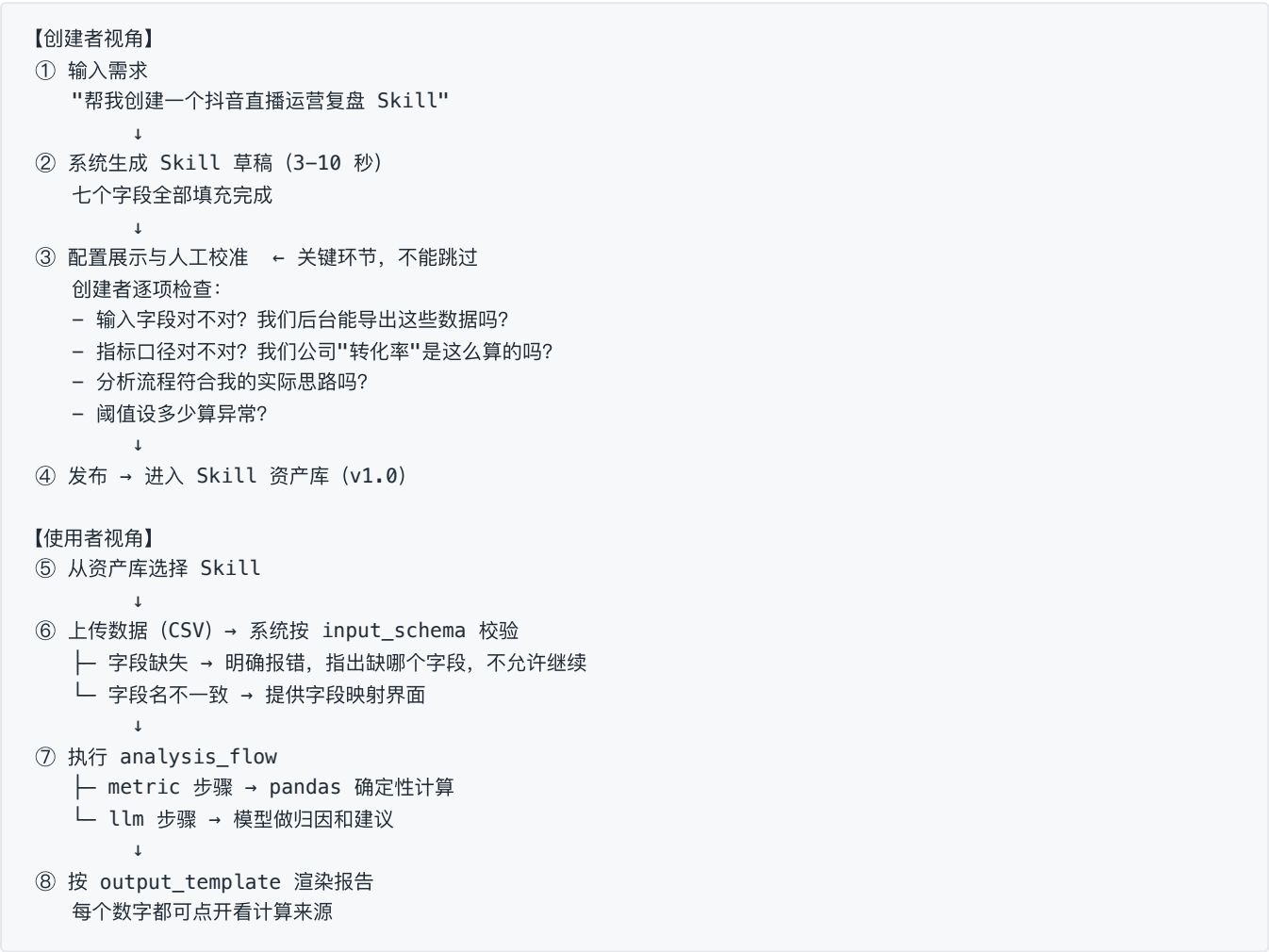
4.3 我们额外补充的两个字段

题目要求了 7 个，我们在实现时补充两个字段，理由是它们直接决定 Skill 能不能"被管理"：

字段	作用
metric_dictionary	指标口径字典。明确"转化率"到底 = 成交订单数/进入人数 还是 /商品点击人数。这是企业内部最容易吵架的地方，必须显式写死
version + status	版本与状态（draft / published / deprecated）。没有版本就没有回滚，没有回滚就不敢改

五、核心用户流程

5.1 主流程：从一句话到一份报告



5.2 异常流程处理（体现产品成熟度）

异常	处理方式
LLM 生成的 JSON 不合法	自动重试 2 次，仍失败则降级为模板化生成 + 提示用户手动补充
生成的 Skill 逻辑不闭环（流程里用了 input 中没有的字段）	校验层直接拦截，标红提示"分析流程 Step3 引用了未定义字段：退款金额"
用户上传数据缺少必填字段	拦截执行，明确列出缺失字段清单，并给出"从抖音后台哪个页面导出"的提示
用户数据字段名与 schema 不一致	自动模糊匹配 + 人工确认的字段映射界面
数据行数过少（如仅 1 场直播但 Skill 需要对比基准）	降级执行：跳过对比类步骤，在报告中明确标注"因缺少基准期数据，本次未做同比分析"

六、Skill 资产如何管理

这是把"玩具 Demo"和"企业级产品"区分开的部分。

6.1 资产管理五要素

① 目录与检索

按业务域（电商/销售/增长/零售）+ 岗位（运营/分析师/店长）+ 分析类型（复盘/诊断/预测）三维打标。
支持自然语言检索"我想找个分析退货率的"。

② 版本管理

- 每次修改产生新版本，旧版本保留可回滚
- 版本状态：草稿 → 待评审 → 已发布 → 已归档
- 已发布版本不可直接编辑，必须新建版本（防止线上行为被静默改变）
- 记录每个版本的 diff、修改人、修改理由

③ 权限与归属

- 每个 Skill 有明确 Owner（通常是创建者所在部门负责人）
- 三级权限：可查看 / 可执行 / 可编辑
- 敏感 Skill（如涉及成本、毛利）按组织架构限制执行范围

④ 生命周期治理

- 上线：需通过评测门槛 + Owner 审批
- 监控：跟踪调用量、执行成功率、用户评分
- 下架：连续 N 天零调用 或 评分低于阈值 → 自动进入待处理队列，提醒 Owner 优化或归档
- 业务变更响应：当底层指标口径变更（如平台改了 GMV 定义），系统标记所有引用该指标的 Skill 为"待复核"

⑤ 复用与继承

- 支持从已有 Skill 复制创建（如"抖音直播复盘"→ 派生"快手直播复盘"）
- 抽取公共指标字典和行动建议库，跨 Skill 共享

6.2 资产健康度看板

管理者视角需要看到的：

指标	说明
Skill 总数 / 已发布数 / 活跃数	资产规模与活性
月调用总量 / Top10 高频 Skill	哪些经验真的在被复用
平均评测分 / 低分 Skill 清单	质量分布
执行成功率 / 失败原因 TOP	主要是数据问题还是 Skill 问题
人工修正率	用户拿到报告后有多少比例做了修改——这是最真实的质量信号
覆盖岗位数 / 未覆盖岗位	资产建设的空白区

七、如何验证 Skill 是否有效

核心观点：不能用"看起来对不对"来验证，必须建立分层验证体系。

7.1 三层验证

第一层：生成期验证（自动，秒级）

发生在 Skill 创建时，防止生成出结构上就不可用的 Skill。

检查项	判定
Schema 完整性	七个字段是否齐全、类型是否正确
逻辑闭环性	analysis_flow 里引用的每个字段，是否都在 input_schema 中定义
指标合法性	用到的指标是否在指标字典白名单内，口径是否明确
输出一致性	output_template 的占位符，是否都能被 analysis_flow 的产出填充
可执行性	用一份 mock 数据能否跑通全流程

第二层：执行期验证（自动，每次运行）

检查项	判定
输入数据校验	必填字段是否齐全、类型是否正确、是否有异常值（如转化率 > 100%）
计算正确性	metric 步骤走确定性代码，结果天然可复现
结论可溯源	报告中每个数字都能追溯到源数据行和计算公式
幻觉检测	扫描 LLM 输出中的数字，是否都出现在上游计算结果中——出现了未经计算的数字即判定为幻觉，拦截并重试

第三层：结果期验证（半自动，周期性）

方法	说明
黄金评测集（Golden Set）	每个 Skill 配 5–10 组"历史数据 + 专家标准答案"。每次 Skill 修改后自动跑回归，对比新版本结论与标准答案的一致性
专家盲评	让业务专家在不知道是 AI 还是人写的情况下，对报告打分
五维评分卡（见下）	结构化打分，可量化、可追踪趋势
业务结果回归	最硬的指标：按 Skill 建议采取行动的场次，后续指标是否真的改善

7.2 Skill 质量评分卡

维度	权重	评价标准	打分方式
数据准确性	30%	报告中的数字是否与源数据一致，无幻觉	自动（比对计算结果）
归因合理性	25%	定位的原因是否符合业务逻辑，是否符合专家判断	专家评 + 与 Golden Set 对比
建议可执行性	25%	建议是否具体到"谁、做什么、什么时候"，而非"建议优化转化率"这种废话	专家评
结构完整性	10%	是否严格遵循 output_template，无缺项	自动
表达清晰度	10%	是否说人话，是否有无关废话	专家评

发布门槛：总分 ≥ 80 分，且数据准确性维度 ≥ 95 分（数字错了一切归零）。

7.3 上线后的持续验证

信号	说明
用户评分 (👍/👎 + 可选理由)	最直接的反馈
人工修正率	用户导出报告后改了多少——修正率越高说明 Skill 越不好用
采纳率	建议被真正执行的比例
复用率	同一用户是否会第二次使用这个 Skill

八、MVP 范围与迭代规划

8.1 MVP（本次 Demo 范围）

目标：跑通完整闭环，验证"自然语言 → 可执行 Skill"这条路走得通。

模块	MVP 包含	MVP 不包含
Skill 生成	自然语言输入 → 七字段生成 → Schema 校验	多轮对话式澄清、语音输入
Skill 配置	可视化展示 + 关键字段可编辑	拖拽式流程编辑器
Skill 存储	本地 JSON/SQLite，带版本号	数据库集群、多租户隔离
数据接入	CSV 上传 + 字段映射	数据库直连、API 对接、实时数仓
Skill 执行	metric 确定性计算 + llm 归因	定时调度、批量执行
结果输出	Markdown 报告 + 图表	PDF 导出、飞书/钉钉推送
质量保障	Schema 校验 + 数字幻觉检测	完整评测集、专家评审 workflow
预置内容	3 个预置 Skill + 配套示例数据	Skill 市场

8.2 后续迭代

阶段	重点	关键能力
V1 质量可信	让企业敢用	评测集 + 回归测试 + 版本管理 + 审批流
V2 数据打通	降低使用门槛	数据源直连 (MySQL/数仓/电商平台 API)、定时执行、异常自动预警
V3 资产运营	规模化复制	Skill 市场、行业模板库、跨 Skill 编排 (多个 Skill 组成一个"AI 员工")
V4 组织协同	成为基础设施	多 Agent 协作、AI 员工绩效看板、与企业 OA/IM 深度集成

九、成功指标

9.1 北极星指标

月度有效 Skill 执行次数 —— 有效 = 执行成功 且 用户未给出负面反馈。

选它的理由：它同时验证了供给侧（有人愿意创建 Skill）和需求侧（有人愿意用），且过滤掉了刷量和无效执行。

9.2 分层指标

层级	指标	说明
供给	累计发布 Skill 数、活跃创建者数	经验有没有在被沉淀
需求	月执行次数、周活使用者数、人均执行次数	经验有没有在被复用
质量	平均评测分、执行成功率、人工修正率、👍 率	沉淀下来的东西靠不靠谱
效率	单次分析耗时 (vs 人工基线)、新人达到熟手水平所需时间	到底省了多少
业务	采纳建议后的指标改善幅度	最终价值证明

十、风险与应对

风险	影响	应对
LLM 生成的 Skill 质量不稳定	用户第一次用就失望，不会有第二次	JSON Schema 强约束 + 指标字典白名单 + 生成后自检 + 强制人工确认环节
业务方不信任 AI 分析结果	做出来没人用	数值走确定性计算 + 结论可溯源到数据行 + 报告中明确标注局限性
数据质量差导致垃圾进垃圾出	结论错误但看起来很像模像样	输入数据强校验 + 异常值检测 + 数据不足时降级执行并明确标注
创建者不愿意贡献经验（怕被替代）	供给端枯竭	产品定位为"专家能力放大器"而非"替代者"；Skill 归属和贡献度可见，与激励挂钩
指标口径在企业内部就不统一	Skill 一发布就被质疑	指标字典作为独立模块前置建设，先统一口径再建 Skill
过度依赖单一大模型	供应商风险、成本风险	模型层抽象，支持多模型可切换；确定性计算部分不依赖模型

十一、非目标（明确不做什么）

明确边界同样重要：

- **✗ 不做 BI 报表工具。**我们不替代看板，我们消费看板的数据做解读
- **✗ 不做通用 Agent 平台。**只聚焦"经营分析"这一类岗位经验
- **✗ 不追求全自动。**人工确认环节是特性不是缺陷——企业级 AI 的信任来自人在回路
- **✗ 不做数据治理。**假设企业已有基本可用的数据，我们不解决数据清洗和数仓建设

企业岗位经验 Skill 生成平台 — 技术方案说明

版本 v1.0 | 关联文档：《01_产品设计文档 PRD》《03_面试问题回答》

一句话技术主张

整个系统围绕一份 **SkillSpec** JSON Schema 展开：LLM 负责把自然语言翻译成 SkillSpec，校验层负责保证 SkillSpec 可执行，执行引擎负责把 SkillSpec 变成分析报告。而报告里的每一个数字，都由确定性代码算出，不由 LLM 生成。

最后这半句是整套方案的信任地基，也是它区别于"套壳 Prompt 生成器"的地方。

一、整体系统架构

1.1 分层架构



1.2 技术选型与理由

层	选型	理由
后端	Python 3.11 + FastAPI	数据分析生态在 Python；FastAPI 自带 Pydantic，天然适合做 SkillSpec 的 Schema 定义与校验；自动生成 OpenAPI 文档
计算	pandas	确定性指标计算的标准选择，可读性强，面试时能逐行解释
Schema	Pydantic v2	一份模型同时用于：API 请求校验、LLM 结构化输出约束、存储序列化。三处复用，避免定义漂移
LLM	Anthropic Claude (claude-opus-5)	结构化输出（output_config.format）能力强，长上下文利于塞入指标字典和 few-shot；通过网关层抽象，可切换其他模型
前端	原生 HTML + Tailwind CSS (CDN) + Alpine.js	无构建步骤、无 npm 依赖，单文件即可运行；Tailwind 保证视觉质量；避免引入不必要的框架复杂度
存储	SQLite (Demo)	零配置、单文件、易于随仓库分发；生产切 PostgreSQL 仅需换 SQLAlchemy 连接串
部署	Railway / Hugging Face Spaces	对 Python 项目零配置部署，免费额度足够 Demo

1.3 目录结构

```
skill-platform/
├── app/
│   ├── main.py                # FastAPI 入口与路由
│   ├── schemas/
│   │   └── skill_spec.py      # ★ SkillSpec Pydantic 模型（全系统的中心）
│   ├── generator/
│   │   ├── prompt.py         # 生成用的 system prompt + few-shot
│   │   ├── generator.py       # 调用 LLM 产出 SkillSpec
│   │   └── validator.py       # ★ 四道校验闸
│   ├── executor/
│   │   ├── loader.py         # CSV 加载与字段映射
│   │   ├── metrics.py        # ★ 确定性指标计算引擎
│   │   ├── orchestrator.py    # analysis_flow 编排
│   │   └── renderer.py        # 按 output_template 渲染报告
│   ├── llm/
│   │   └── client.py          # LLM 网关（多模型抽象）
│   ├── knowledge/
│   │   ├── metric_dictionary.yaml # 指标口径白名单
│   │   └── action_library.yaml   # 行动建议库
│   ├── evaluation/
│   │   └── scorer.py          # 评分卡与幻觉检测
│   └── storage/
│       └── repository.py       # Skill 资产 CRUD + 版本管理
├── static/
│   └── index.html              # 单页前端
├── data/
│   ├── skills.db              # SQLite
│   ├── presets/               # 3 个预置 Skill
│   └── samples/               # 配套示例 CSV
├── tests/
│   └── golden_set/            # 评测集
├── requirements.txt
└── README.md
```

二、核心数据结构：SkillSpec

这是整个系统的中心。所有模块都围绕它读写。

```

from pydantic import BaseModel, Field
from typing import Literal, Optional
from enum import Enum

class FieldType(Enum):
    STRING = "string"
    INTEGER = "integer"
    FLOAT = "float"
    DATE = "date"
    ENUM = "enum"

class InputField(BaseModel):
    """输入数据定义 — 数据契约"""
    name: str # 字段名, 如 "曝光人数"
    type: FieldType
    required: bool = True
    unit: Optional[str] = None # 单位, 如 "人" "元" "秒"
    description: str # 口径说明, 如 "直播间被展示的去重人数"
    source_hint: Optional[str] = None # 从哪导出, 如 "抖音罗盘-直播分析-流量看板"

class MetricStep(BaseModel):
    """确定性计算步骤 — 由代码执行, 不经过 LLM"""
    type: Literal["metric"] = "metric"
    step_id: str
    name: str # "计算核心转化漏斗指标"
    formula: str # "进入率 = 进入直播间人数 / 曝光人数"
    inputs: list[str] # 依赖的输入字段
    outputs: list[str] # 产出的指标名

class LLMStep(BaseModel):
    """模型推理步骤 — 只做归因、解读、建议, 不做算数"""
    type: Literal["llm"] = "llm"
    step_id: str
    name: str # "定位下滑环节并归因"
    instruction: str # 给模型的具体指令
    inputs: list[str] # 依赖的上游产出 (指标名或前序步骤 output)
    outputs: list[str]

AnalysisStep = MetricStep | LLMStep

class SkillSpec(BaseModel):
    # ===== 题目要求的 7 个字段 =====
    name: str # ① Skill 名称
    description: str # ② Skill 描述
    use_cases: list[str] # ③ 使用场景
    input_schema: list[InputField] # ④ 输入数据定义
    analysis_flow: list[AnalysisStep] # ⑤ 分析流程
    agent_prompt: str # ⑥ Agent Prompt
    output_template: str # ⑦ 输出结果模板 (Markdown + 占位符)

    # ===== 治理所需的补充字段 =====
    skill_id: str
    version: str = "1.0.0"
    status: Literal["draft", "published", "deprecated"] = "draft"
    domain: str # 业务域: 电商/销售/增长/零售
    owner: Optional[str] = None
    metric_dictionary: dict[str, str] = {} # 指标口径显式定义

```

```
created_at: str
updated_at: str
```

为什么 `analysis_flow` 要区分 `metric` 和 `llm` 两种步骤？

这是全套方案最关键的设计决策。详见第五节。

三、LLM 在系统中承担什么角色

一句话：LLM 是"翻译官"和"分析师"，不是"计算器"和"裁判"。

环节	LLM 是否参与	承担什么	为什么这样划分
需求理解	✅ 主力	把"帮我做个直播复盘 Skill"扩展成完整的业务理解：需要哪些数据、专家会怎么看	这正是 LLM 的强项——把模糊需求映射到领域知识
Skill 结构化生成	✅ 主力	输出符合 SkillSpec Schema 的 JSON	用 structured output 强约束，把创造力限制在正确的形状里
生成自检	✅ 辅助	对自己生成的 Skill 做批判性检查（self-critique）	模型评价比模型生成更可靠，是低成本的质量提升
Schema 校验	❌	——	结构正确性是确定性问题，用代码判定，不能交给概率模型
指标数值计算	❌ 绝对不参与	——	LLM 算数不可靠，且不可复现。企业不会信任一个自己算出来的数字
异常环节定位	⚠️ 混合	阈值判断走代码，业务解读走 LLM	"转化率下降 15% 是否显著"是规则问题；"为什么会下降"是理解问题
归因分析	✅ 主力	结合业务上下文解释指标变化的原因	这是 LLM 不可替代的价值
行动建议生成	✅ 主力	从行动建议库中匹配并具体化为可执行动作	需要理解语境做个性化表达
报告文本组织	✅ 主力	按 output_template 组织成人话	——
最终结论审批	❌	——	高风险决策必须人在回路

3.1 LLM 网关抽象

```
class LLMGateway:
    """屏蔽模型差异，支持切换与降级"""

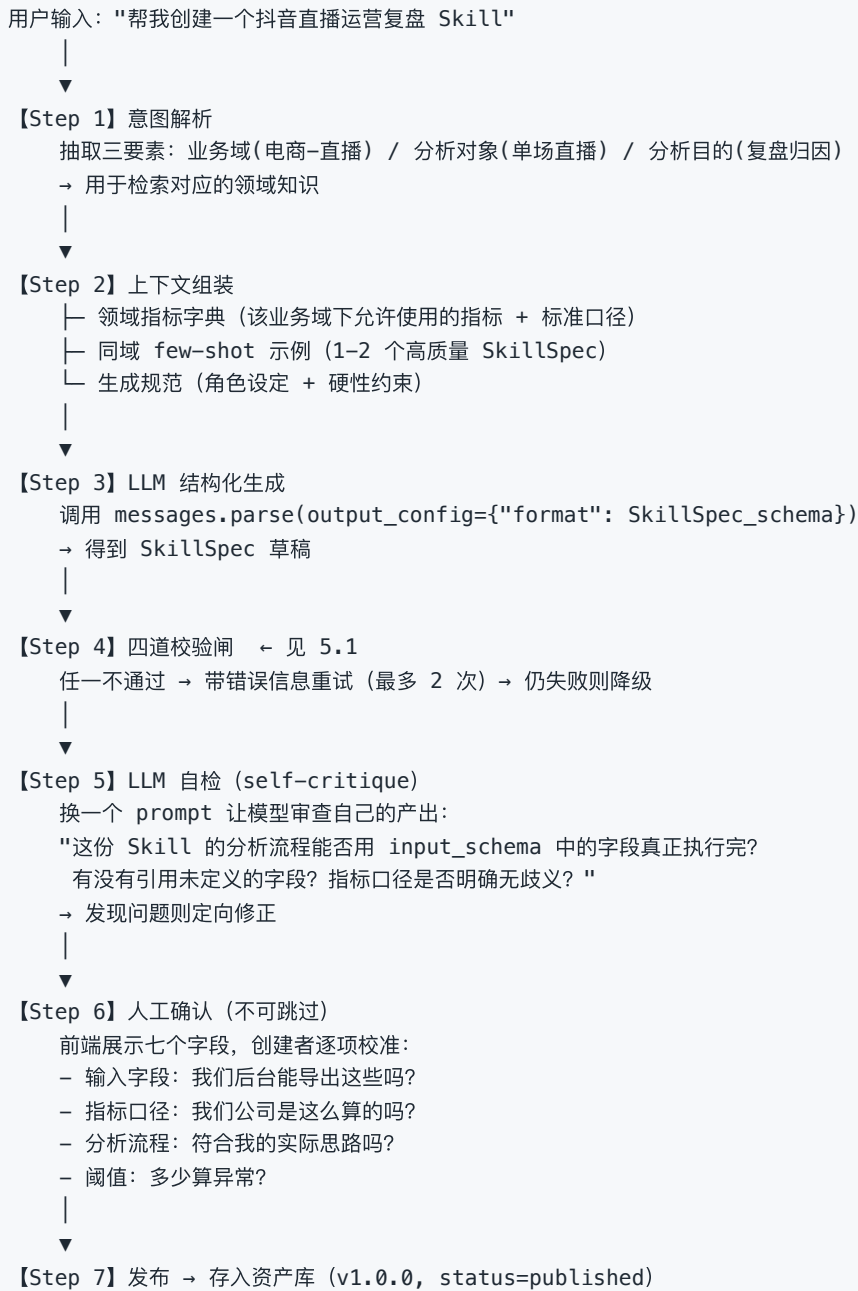
    def generate_structured(
        self,
        system: str,
        user: str,
        schema: type[BaseModel],  # Pydantic 模型
        max_retries: int = 2,
    ) -> BaseModel:
        """结构化输出：保证返回值符合 schema"""

    def generate_text(self, system: str, user: str) -> str:
        """自由文本输出：用于归因和建议"""
```

Demo 使用 Claude 的 `client.messages.parse()` + Pydantic 模型，由 API 层面保证 JSON 合法性；切换到其他模型时，网关内部改用 JSON mode + 手动校验重试即可，上层无感知。

四、如何生成 Skill（生成链路详解）

4.1 完整流程



4.2 生成 Prompt 的关键设计

System Prompt 骨架:

你是企业经营分析领域的 Skill 架构师。你的任务是把业务专家的岗位经验，转换为一份结构化、可执行的 SkillSpec。

- 【硬性约束】
- analysis_flow 中的每一个字段引用，必须在 input_schema 中已定义。不允许出现"凭空冒出来"的字段。
 - 所有涉及数值计算的步骤，type 必须为 "metric"，并给出明确的 formula。不允许把计算写进 llm 步骤的 instruction 里。
 - 只允许使用【指标字典】中已定义的指标。若业务确实需要新指标，在 metric_dictionary 中显式给出口径定义。
 - input_schema 中的每个字段必须给出 description（口径说明）和 source_hint（数据从哪导出），否则使用者无法准备数据。
 - agent_prompt 必须包含【禁止事项】章节，至少包含：
 - 不得自行计算任何数值，所有数字引用上游 metric 步骤的产出
 - 数据不足时明确指出缺失项，不得臆测

【分析流程的编写原则】
按业务专家的真实思考路径组织，通常遵循：
测量（算出核心指标）→ 对比（与基准比）→ 定位（找到问题环节）
→ 归因（解释为什么）→ 行动（给出怎么办）

【可用指标字典】
{metric_dictionary}

【参考示例】
{few_shot_examples}

为什么强调这些约束？ 因为不加约束时，LLM 最容易犯三个错：把计算塞进 Prompt 里让模型自己算、引用不存在的字段、指标口径含糊（"转化率"不说清楚是除以什么）。这三个错任何一个都会让 Skill 无法执行或结果不可信。

4.3 降级策略

失败情形	降级方案
结构化输出连续 2 次不合法	切换为"分字段生成"：把七个字段拆成七次小请求，降低单次复杂度
分字段仍失败	返回同业务域模板 Skill + 提示用户手动编辑
LLM 服务不可用	直接返回模板库，功能降级但不中断

五、如何避免 AI 生成错误的 Skill

这是本方案最重要的部分。防错不靠"让模型更聪明"，靠机制。

5.1 四道校验闸（生成期）



5.2 结构性防错设计

除了校验闸，架构本身就在防错：

① 指标字典白名单 —— 禁止 LLM 自创口径

```
# knowledge/metric_dictionary.yaml
电商直播：
  进入率：
    formula: "进入直播间人数 / 曝光人数"
    unit: "%"
    healthy_range: [0.05, 0.20]
    description: "衡量直播间封面和标题的吸引力"
  商品点击率：
    formula: "商品点击人数 / 进入直播间人数"
    unit: "%"
    healthy_range: [0.10, 0.35]
  成交转化率：
    formula: "成交订单数 / 商品点击人数"
    unit: "%"
    note: "注意：本口径为点击成交率，非全场转化率"
  UV价值：
    formula: "GMV / 进入直播间人数"
    unit: "元"
```

企业内部最容易吵架的就是口径。把口径抽出来做成独立的、可治理的白名单，LLM 只能引用不能发明，这从源头消灭了"AI 算的转化率和我算的不一样"这类争议。

② 数值与语言分离 —— 从架构上消灭数字幻觉

这是最核心的设计：

- ✗ 错误做法（大多数"套壳"方案）
 - 把整张 CSV 和一段 Prompt 全部塞给 LLM，让它"分析一下"
 - 数字是模型生成的，可能算错、可能编造，且不可复现
- ✓ 本方案
 - metric 步骤 → pandas 确定性计算 → 得到一份【指标结果表】
 - llm 步骤 → 只接收【指标结果表】，不接触原始数据
 - 指令中明确"你不需要也不允许计算任何数字，所有数值必须直接引用给你的结果表"
 - 数字 100% 正确且可复现，模型只负责"解释"和"建议"

③ 人在回路（Human-in-the-loop）

生成后必须经过创建者确认才能发布。这不是"AI 不够强"的妥协，而是企业级产品的必要设计——责任必须有人承担。

④ 版本化与灰度

- 已发布版本不可直接编辑，改动必产生新版本
- 新版本可先小范围灰度，观察评测分和用户反馈后再全量
- 出问题一键回滚到上一版本

5.3 执行期防错

机制	说明
输入数据校验	必填字段缺失 → 拦截执行并列出缺失清单；类型不符 → 报错；数值异常（转化率 > 100%、负数 GMV） → 警告并标注
计算异常防护	除零保护、空值处理、样本量不足时跳过对比类步骤并在报告中明确标注
数字幻觉检测	用正则扫描 LLM 输出中的所有数字，与上游 metric 结果集比对；出现结果集中不存在的数字 → 判定幻觉 → 重试；连续失败则在报告中标红提示
溯源信息保留	每个指标记录 {值, 公式, 依赖字段, 源数据行数}，前端可点开查看

六、如何实现 Skill 执行

6.1 执行流程

用户选择 Skill + 上传 CSV

|

▼

【1】数据加载

`pandas.read_csv()` → DataFrame

|

▼

【2】字段映射

自动模糊匹配 CSV 列名 ↔ input_schema 字段名
("曝光人数" ↔ "曝光量" ↔ "impressions")

→ 展示映射结果供用户确认/调整

|

▼

【3】input_schema 校验

├ 必填字段缺失 → 中断, 返回缺失清单 + source_hint 提示

├ 类型不符 → 尝试转换, 失败则报错

└ 异常值 → 记录 warning, 继续执行

|

▼

【4】流程编排 (orchestrator)

构建 DAG, 按依赖顺序执行 analysis_flow:

```
context = {"raw_data": df, "metrics": {}, "llm_outputs": {}}
```

```
for step in analysis_flow:
```

```
    if step.type == "metric":
```

```
        result = MetricEngine.execute(step.formula, context)
```

```
        context["metrics"][step.outputs] = result
```

```
        # 同时记录溯源信息
```

```
    elif step.type == "llm":
```

```
        # ★ 只传指标结果, 不传原始数据
```

```
        payload = extract(context, step.inputs)
```

```
        result = llm.generate_text(
```

```
            system=skill.agent_prompt,
```

```
            user=render(step.instruction, payload)
```

```
        )
```

```
        hallucination_check(result, context["metrics"]) # 幻觉检测
```

```
        context["llm_outputs"][step.outputs] = result
```

|

▼

【5】报告渲染

按 output_template 填充占位符 → Markdown

数值部分附带溯源标记, 前端可展开查看计算过程

|

▼

【6】持久化执行记录

存入 execution_log: skill_id, version, 输入数据摘要,

各步骤耗时, token 消耗, 结果, 用户反馈

6.2 指标计算引擎

安全地执行 `formula` 字符串是个关键点。不能用 `eval()`（任意代码执行风险）。

方案：受限表达式解析器

```
class MetricEngine:
    """
    只支持四则运算、比较、聚合函数的受限求值器。
    基于 Python ast 模块解析表达式树，白名单节点类型。
    """
    ALLOWED_FUNCS = {"sum", "mean", "max", "min", "count", "abs", "round"}
    ALLOWED_NODES = (ast.Expression, ast.BinOp, ast.Num, ast.Name,
                     ast.Call, ast.Compare, ast.Constant, ...)

    def execute(self, formula: str, context: dict) -> MetricResult:
        tree = ast.parse(formula, mode="eval")
        self._validate_nodes(tree)      # 白名单校验，遇到非法节点直接拒绝
        value = self._eval(tree, context)
        return MetricResult(
            value=value,
            formula=formula,
            depends_on=self._extract_names(tree),
            row_count=len(context["raw_data"]),
        )
```

6.3 关键实现细节

问题	处理
CSV 编码/分隔符不确定	<code>chardet</code> 探测编码，尝试多种分隔符
数值列含千分位、百分号、货币符号	预处理清洗后转 float
单行数据 vs 多行数据	Skill 声明 <code>granularity</code> （单次/时序），编排器按声明选择聚合方式
LLM 调用超时	设置 timeout + 重试；步骤级失败不中断全流程，在报告中标注该章节失败
长报告的 token 成本	metric 结果已高度压缩（几十个数字），llm 步骤输入很小，成本可控

七、如何实现 Skill 评估

7.1 三层评估体系

层	时机	方式	判定
L1 生成期	Skill 创建时	四道校验闸（见 5.1）	自动，硬门槛，不通过不能保存
L2 执行期	每次运行	输入校验 + 幻觉检测 + 溯源完整性	自动，实时
L3 结果期	发布前 + 周期性	黄金评测集回归 + 多维评分卡 + 专家盲评	半自动，决定能否发布/是否下架

7.2 黄金评测集（Golden Set）

这是让 Skill 从"能跑"变成"可信"的关键。

```
tests/golden_set/  
└─ douyin_live_review/  
   │ └─ case_01_traffic_drop/  
   │   │ └─ input.csv           # 一场真实的流量下滑直播数据  
   │   │ └─ expected.yaml       # 专家标注的标准答案  
   │   │ └─ metrics_truth.json  # 人工核算的正确指标值  
   │ └─ case_02_conversion_drop/  
   │ └─ case_03_normal/         # 正常场次，检验是否会误报  
   └─ case_04_missing_data/     # 数据缺失，检验降级行为
```

expected.yaml 示例：

```
expected_metrics:           # 硬指标，必须完全一致  
  进入率: 0.082  
  商品点击率: 0.156  
  成交转化率: 0.043  
expected_diagnosis:        # 关键结论，必须命中  
  bottleneck_stage: "流量层"  
  root_cause_keywords: ["自然流量下滑", "推荐权重", "封面点击率"]  
expected_actions:          # 建议方向，命中 ≥ 2 条即通过  
  - "优化直播间封面与标题"  
  - "检查违规扣分情况"  
  - "调整开播时段"  
must_not_contain:          # 反例，出现即判失败  
  - "建议提升转化率"      # 空话，无可执行性
```

7.3 自动评分实现

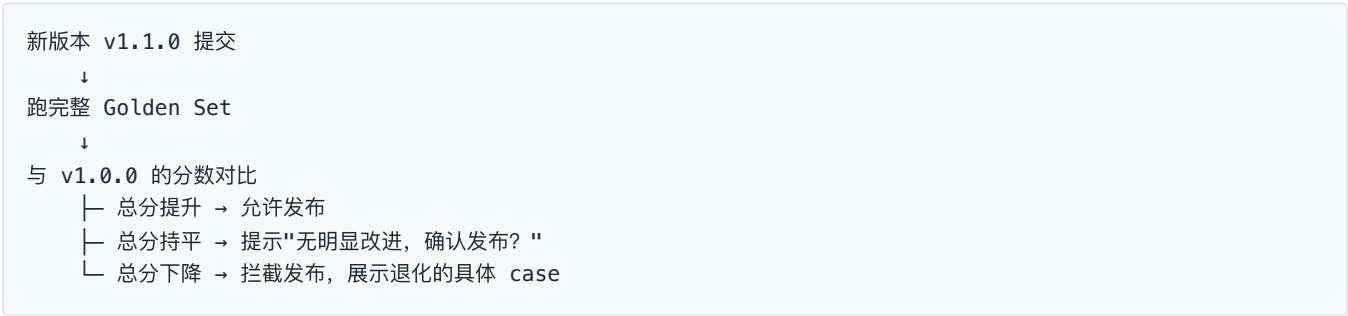
```
def evaluate(skill: SkillSpec, case: GoldenCase) -> ScoreCard:
    report = execute_skill(skill, case.input_csv)

    return ScoreCard(
        # 30% — 全自动，数值精确比对
        data_accuracy = exact_match(
            report.metrics, case.expected_metrics, tol=1e-6
        ),
        # 25% — 半自动，用 LLM-as-judge 判断结论是否等价
        attribution = llm_judge(
            report.diagnosis, case.expected_diagnosis
        ),
        # 25% — 半自动，关键词命中 + LLM 判断可执行性
        actionability = keyword_hit(report.actions, case.expected_actions)
            * llm_judge_actionable(report.actions),
        # 10% — 全自动
        completeness = template_conformance(report, skill.output_template),
        # 10% — LLM-as-judge
        clarity = llm_judge_clarity(report),
    )
```

发布门槛：加权总分 ≥ 80 ，且 `data_accuracy` ≥ 95 。
数据准确性一票否决——数字错了，其他维度再高也没有意义。

7.4 回归测试

每次 Skill 修改（产生新版本）时自动触发：



这套机制让 Skill 的迭代变成工程化的、可度量的过程，而不是"改了 Prompt 感觉好像好点了"。

7.5 线上持续评估

信号	采集方式	用途
用户评分	报告页 🍌/🍌 + 可选理由	直接质量信号
人工修正率	对比用户导出前后的报告 diff	最真实的质量信号——改得越多说明越不好用
执行成功率	执行日志统计	区分是 Skill 问题还是数据问题
复用率	同用户 30 天内重复使用比例	判断是否真的解决了问题

低分 Skill 自动进入 Owner 待办队列，触发优化或归档。

八、Demo 实现范围与验收

8.1 Demo 交付清单

模块	实现内容
SkillSpec Schema	完整 Pydantic 模型
生成链路	自然语言输入 → LLM 结构化生成 → 四道校验闸 → 前端展示
Skill 配置展示	七字段可视化，关键字段可编辑
Skill 资产库	SQLite 存储，含 3 个预置 Skill + 版本号
执行链路	CSV 上传 → 字段映射 → 校验 → metric 计算 → llm 归因 → 报告渲染
幻觉检测	数字比对，异常标红
溯源展示	报告中数值可点开查看公式与依赖
评测	至少 1 个 Skill 配 3 组 Golden Case，可一键跑评分

8.2 三个预置 Skill

Skill	业务域	演示重点
抖音直播运营复盘	电商直播	完整漏斗分析，展示 metric/llm 分离
销售漏斗健康度诊断	销售	时序对比，展示基准期逻辑
电商 GMV 波动归因	电商	多维拆解，展示复杂 analysis_flow

8.3 演示脚本（3 分钟讲清价值）

- 输入"帮我创建一个抖音直播运营复盘 Skill" → 展示生成过程与七字段结果
- 故意展示校验拦截**：手动把 analysis_flow 改成引用一个不存在的字段 → 系统精确报错
- 上传示例 CSV → 展示字段映射与校验
- 执行 → 展示报告，**点开某个数字看溯源**（这是最有说服力的一幕）
- 展示 Golden Set 评分结果
- 展示版本管理与资产库

九、技术风险与应对

风险	应对
LLM 结构化输出不稳定	Claude structured output + Pydantic 双重保障 + 重试 + 分字段降级
<code>formula</code> 字符串执行的安全风险	ast 白名单解析, 禁用 <code>eval</code> / <code>exec</code>
用户 CSV 格式千奇百怪	编码探测 + 分隔符探测 + 字段模糊匹配 + 人工确认映射界面
LLM 成本失控	metric 结果高度压缩后再传给 LLM; 指标字典和 few-shot 用 prompt caching
单一模型供应商依赖	LLM 网关抽象层, 配置切换; 确定性计算部分完全不依赖模型
Demo 环境无 API Key 时无法演示	内置 mock 模式: 预置生成结果与执行结果, 无 Key 也能跑通全流程演示

面试问题回答

对应题目第六部分 | 关联文档：《01_产品设计文档 PRD》《02_技术方案说明》

Q1. 为什么企业需要 Skill，而不是直接使用 ChatGPT？

因为 ChatGPT 解决的是"个人能力放大"，而企业需要的是"组织能力沉淀"。这是两个不同层次的问题。

ChatGPT 让一个人变强，但它不会让组织变强——因为每个人的用法都不一样，而且这些用法留不下来。

具体拆开说，企业场景下缺了五样东西

① 缺一致性——结果不可复现

同一个问题，A 用 ChatGPT 问和 B 问，得到的分析框架不一样；今天问和明天问，结论可能也不一样。这在闲聊场景无所谓，但在经营分析场景是致命的——老板不可能接受"上周说是流量问题，这周同样的数据说是转化问题"。

企业需要的是：同样的数据，任何人、任何时候执行，得到同样口径下的同样结论。

② 缺数据契约——不知道该喂什么

ChatGPT 是"你给什么它分析什么"。但一线员工恰恰不知道该看哪些数据——这本身就是专家经验的一部分。Skill 的 `input_schema` 把"分析这件事需要哪 12 个字段、每个字段什么口径、从后台哪个页面导出"固化下来，这是 ChatGPT 天然给不了的。

③ 缺数值可信度——数字是编的

把一张 CSV 丢给 ChatGPT 让它算转化率，它会给你一个数——但这个数是模型"生成"出来的，不是"算"出来的。它可能对，可能错，而且你无法验证、无法复现。企业拿这种数字做决策是不可接受的。

Skill 的做法是：数值走确定性代码，模型只做解读。

④ 缺资产化——经验留不下来

用 ChatGPT 的过程发生在个人的聊天记录里。人走了，聊天记录也带走了；就算留下，也没人知道哪条对话是有价值的。Skill 是有归属人、有版本、有评测分、可被检索和授权的**资产**，它进的是公司的资产库而不是个人的聊天框。

⑤ 缺治理——不敢用、不可审计

企业级 AI 要回答的问题是：谁能用？用了什么数据？结论怎么来的？出错了怎么回滚？谁负责？

这些 ChatGPT 一个都答不了。Skill 平台通过权限、版本、执行日志、溯源信息把这些补齐。

总结

ChatGPT 是给个人的通用工具，Skill 是组织的专用资产。

企业买的不是"能对话的 AI"，是"能被管理、能被信任、能被复用的分析能力"。

补充：那 ChatGPT 在这个方案里是什么角色？

不是竞争关系。大模型是 Skill 平台的底层能力供应商——我们用它来做需求理解、结构化生成、归因分析。它是发动机，Skill 平台是整车。企业买的是车，不是发动机。

Q2. 你认为一个企业岗位 Skill 最重要的组成部分是什么？

如果只能保留一个，我选「分析流程」(`analysis_flow`)。

理由：其他六个字段描述的是"这个 Skill 是什么"，只有分析流程描述的是"专家是怎么想的"。而后者才是真正稀缺的、不可替代的岗位经验。

为什么是它

名称、描述、使用场景——这些是元数据，任何人花五分钟都能写。

Agent Prompt、输出模板——这些是表达层，可以复用通用模板。

输入数据定义——重要，但它其实是分析流程的**衍生结果**：因为流程里要算进入率，所以才需要曝光人数和进入人数这两个字段。

只有分析流程，编码了专家真正的思考路径：

- **先看什么、后看什么**——为什么先看流量再看转化，而不是反过来？因为漏斗有先后，上游没问题才轮到下游
- **怎么判断异常**——降 5% 是正常波动还是要报警？这个阈值是专家用三年经验试出来的
- **发现问题后往哪个方向拆**——流量掉了，是拆渠道还是拆时段？专家知道哪个方向的信息量更大
- **不同结论对应什么动作**——这是从"分析"到"经营"的最后一公里，也是最值钱的一段

新人和专家的差距，从来不在"会不会算转化率"，而在"该看哪个转化率、看完之后怎么办"。这个差距就装的分析流程里。

但我想补充一个观点

如果问"最重要的"，答案是分析流程；如果问"最容易被低估的"，我的答案是「输入数据定义」(`input_schema`)。

因为它是**决定 Skill 能不能真正落地的那个字段**。

分析流程再精妙，如果它要求的数据企业根本导不出来，这个 Skill 就是废的。我在设计 `input_schema` 时特意加了 `source_hint` 字段（"从抖音罗盘-直播分析-流量看板导出"），就是因为：**很多 AI 产品死在"用户不知道该给什么数据"这一步，而不是死在分析能力不够。**

一个不能被执行的完美 Skill，价值是零。

总结

分析流程是 Skill 的灵魂，输入数据定义是 Skill 的地基。
前者决定它有多聪明，后者决定它能不能站起来。

Q3. 如果让你设计一个"销售分析 AI 员工"，需要采集哪些专家经验？

我会把采集工作结构化成五类，因为专家的经验不是一团模糊的"感觉",而是可以被拆解的五种确定的东西。

采集方式：**不问"你有什么经验"**（他答不上来），而是拿三个真实案例让他复盘——一个业绩达标的月份、一个业绩崩了的月份、一个数据看起来正常但他觉得有问题的月份。让他边看边说，我在旁边记录他每一次"为什么看这个"。

五类经验

第一类：指标体系与口径（Metrics）

这是地基，也是最容易在企业内部吵架的地方。

要采集的	具体问题
核心结果指标	你判断一个销售/团队好不好，第一眼看哪三个数？
过程指标	结果出来之前，哪些先行指标能预警？（新增商机数、跟进频次、关键节点推进速度）
口径细节	"成单"是签合同算还是回款算？"商机"从哪个阶段开始算？跨月的订单算哪个月？
效率指标	人均产能、单均金额、销售周期长度、获客成本
健康度指标	漏斗各阶段转化率、商机停留时长、赢单率、流失率

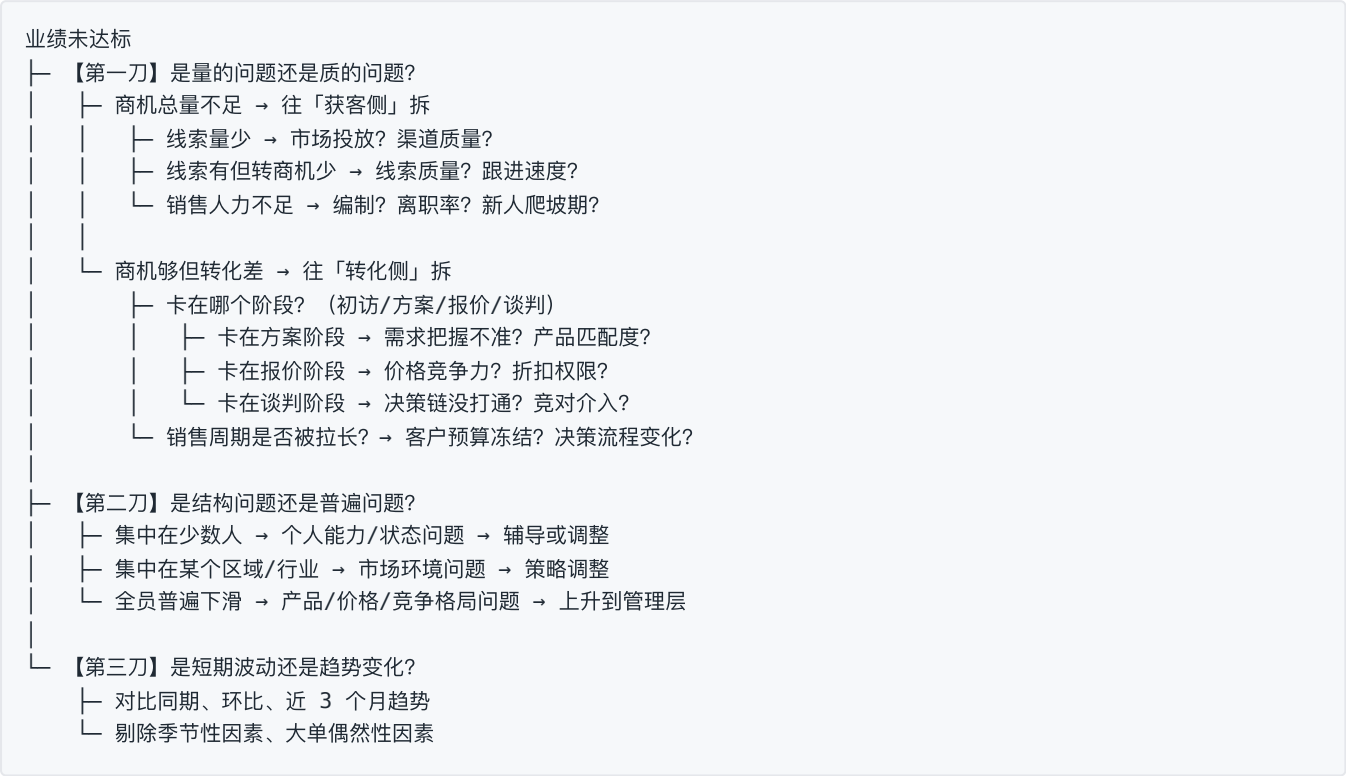
这里最关键的是口径。我见过太多"数据打架"的场景，本质都是口径不统一。所以在我的方案里，指标口径被抽成独立的 `metric_dictionary` 白名单，先统一口径，再建 Skill。

第二类：分析路径与归因树（Diagnosis Logic）

这是最核心、也最难采集的部分——专家的思考顺序。

采集方法：让他对着一个"业绩掉了 30%"的案例，把每一步判断说出来。

典型的销售分析归因树长这样：



要问的关键问题：

- 你为什么先看量再看质，而不是反过来？
- 什么情况下你会跳过某一步？
- 有没有哪次你按这个路径判断错了？错在哪？

第三类：阈值与基准（Thresholds & Benchmarks）

专家脑子里有一套"什么算正常"的标尺，新人没有。

要采集的	举例
正常波动范围	月度业绩上下浮动多少以内不用管？（比如 ±15%）
报警线	赢单率低于多少必须介入？商机停留超过几天算卡住？
分层基准	新人（<3个月） / 成熟销售 / Top Sales 的指标基准分别是多少？
行业/区域差异	不同行业客户的销售周期基准差多少？
季节性规律	哪几个月天然是淡季？春节前后怎么调整预期？

阈值是最容易被忽略但最能体现专业度的部分。没有阈值的分析只能说"下降了 12%", 有阈值的分析才能说"下降 12% 已超出正常波动范围, 需要介入"。

第四类：行动建议库（Action Library）

从"分析"到"经营"的最后一公里。这是让 AI 员工从"分析师"变成"顾问"的关键。

采集格式：归因结论 → 对应动作 → 责任人 → 预期效果 → 见效周期

归因结论	建议动作	责任人	预期效果	周期
线索转商机率低于基准	检查线索响应时长；建立 2 小时首响机制	销售主管	转化率 +5~8pt	2 周
商机集中卡在报价阶段	复盘近 20 个流失单的报价与竞对价差；评估折扣权限下放	销售负责人	报价阶段通过率提升	1 个月
单个销售业绩连续 2 月低于团队均值 50%	一对一复盘其在手商机；判断是能力问题还是分配问题	直属主管	——	1 周
某行业赢单率显著低于其他行业	评估该行业产品匹配度；考虑调整资源投放优先级	销售负责人 + 产品	——	1 个季度

采集时必须追问的： "如果是这个原因，你会让谁去做什么？ "——很多专家能诊断但说不出动作，要逼他具体到人和动作。

第五类：陷阱与例外（Pitfalls）

这一类最容易被忽略，但它是区分"AI 看起来专业"和"AI 真的专业"的地方。

类型	举例
数据陷阱	大单会严重扭曲人均值，要看中位数；CRM 里"已赢单"可能只是销售提前标记的
归因陷阱	业绩下滑正好赶上系统迁移导致数据缺失，不是真的下滑
时间陷阱	长周期业务的当月业绩反映的是 3 个月前的动作，不能用当月动作解释当月业绩
幸存者偏差	只分析成单客户的特征，会漏掉那些同样有这些特征但没成单的
组织例外	新人前 3 个月不参与排名；战略客户不看短期 ROI

采集问题： "有没有哪次数据看起来很清楚，但结论其实是错的？ "

采集方法论：从访谈到 Skill

【1】案例复盘访谈（2-3 小时 × 2 位专家）
准备三个真实案例（好/坏/看似正常但有问题）
让专家边看边说，全程录音
重点记录每一次"为什么看这个"和"这个数说明什么"

↓

【2】结构化拆解
把录音内容归入上述五类
对模糊表述追问到可判定（"感觉不太对" → "低于多少算不对"）

↓

【3】交叉验证
找第二位专家审阅第一位的产出
分歧点单独讨论—分歧往往暴露了口径不统一

↓

【4】转写为 SkillSpec
指标体系 → input_schema + metric_dictionary
归因树 → analysis_flow
阈值 → metric 步骤的判定逻辑
行动库 → llm 步骤的知识注入
陷阱 → agent_prompt 的【禁止事项】和【注意事项】

↓

【5】盲测校准
拿 5 个专家没见过的历史案例跑 Skill
对比 AI 结论与专家结论 → 差异点回炉修正
这 5 个案例同时成为 Golden Set

总结

采集专家经验的难点不是"他不愿意说",而是"他说不清楚"——因为那些判断对他已经是肌肉记忆。所以不能用访谈问卷，要用**案例复盘 + 追问到可判定**。

判断采集是否到位的标准只有一个：**这套东西能不能让一个新人做出和专家一样的判断。**

Q4. 如何判断 AI 生成的 Skill 是否达到企业使用标准？

不能靠"看起来对不对"来判断。我设计了一套三层验证 + 多维评分卡的体系，每一层解决不同的问题。

三层验证：从"能跑"到"跑得对"到"跑得有用"

层级	解决什么问题	时机	方式
L1 结构可用	这个 Skill 在技术上能不能跑起来	生成时	全自动，硬门槛
L2 结果正确	跑出来的数字对不对、有没有幻觉	每次执行	全自动，实时
L3 业务有效	跑出来的结论有没有用	发布前 + 周期性	半自动 + 专家

L1 结构可用性验证（四道校验闸）

闸	检查什么	举例
① 结构校验	七字段齐全？类型正确？	Pydantic Schema 强制校验
② 逻辑闭环	流程里引用的字段是否都已定义？输出模板的占位符是否都有产出？	拦截"Step3 引用了未定义字段：退款金额"
③ 领域规则	指标是否在白名单内？口径是否明确？公式是否可解析？	拦截 LLM 自创的"用户满意度指数"
④ 冒烟测试	用 mock 数据能否真的跑通全流程？	拦截除零、类型错误等运行时问题

任一不通过，Skill 不允许保存。这一层是纯工程问题，必须 100% 自动化。

L2 结果正确性验证

检查项	方式
输入数据合法性	必填字段、类型、异常值（转化率 > 100%、负数 GMV）
计算正确性	metric 步骤走确定性代码，天然可复现——这一项在架构上就保证了
数字幻觉检测	扫描 LLM 输出中的所有数字，与上游计算结果比对；出现结果集中不存在的数字即判定幻觉，拦截重试
溯源完整性	每个数字能否追溯到公式和源数据行

L3 业务有效性验证

核心工具：黄金评测集（Golden Set）

每个 Skill 配 5-10 组「历史真实数据 + 专家标注的标准答案」，覆盖四类场景：

- 典型问题场景（如流量下滑）
- 另一类典型问题（如转化下滑）
- 正常场景（检验会不会误报）
- 数据缺失场景（检验降级行为是否正确）

```
# 标准答案示例
expected_metrics:          # 硬指标，必须完全一致
  进入率: 0.082
expected_diagnosis:        # 关键结论，必须命中
  bottleneck_stage: "流量层"
  root_cause_keywords: ["自然流量下滑", "推荐权重"]
expected_actions:          # 建议方向，命中 ≥ 2 条通过
  - "优化直播间封面与标题"
must_not_contain:          # 反例，出现即失败
  - "建议提升转化率"      # 空话，无可执行性
```

五维评分卡与发布门槛

维度	权重	评价标准	打分方式
数据准确性	30%	数字与源数据一致，无幻觉	全自动（精确比对）
归因合理性	25%	原因符合业务逻辑与专家判断	LLM-as-judge + 专家抽检
建议可执行性	25%	具体到"谁、做什么、什么时候"，不是空话	关键词命中 + 专家评
结构完整性	10%	严格遵循输出模板，无缺项	全自动
表达清晰度	10%	说人话，无废话	LLM-as-judge

发布门槛：加权总分 ≥ 80 分，且数据准确性 ≥ 95 分。

数据准确性一票否决——**数字错了，其他四个维度得分再高也没有意义**，因为企业根本不会用一份数字有错的报告。

回归测试：让迭代变成工程行为

每次 Skill 修改产生新版本时自动触发：

```
新版本提交 → 跑完整 Golden Set → 与旧版本分数对比
├─ 提升    → 允许发布
├─ 持平    → 提示"无明显改进，确认发布？"
└─ 下降    → 拦截发布，展示退化的具体 case
```

这一步让 Skill 的优化从"改了 Prompt，感觉好像好点了"变成可度量的工程过程。

上线后：真实使用数据才是终审

信号	为什么重要
人工修正率	用户导出报告后改了多少内容—— 这是最诚实的质量信号 ，改得越多说明越不好用
采纳率	建议被真正执行的比例
复用率	同一用户 30 天内是否会用第二次
👍/👎 评分	直接反馈

低分 Skill 自动进入 Owner 待办队列，触发优化或归档。

总结

判断标准分三层：**能不能跑（自动）、跑得对不对（自动）、跑得有没有用（专家 + 数据）**。
前两层必须 100% 自动化且是硬门槛，第三层靠评测集把主观判断变成可量化的分数。
而最终的裁判是线上的人工修正率——**用户改得越少，说明 Skill 越好**。

Q5. 如果 AI 员工分析结果错误，企业为什么应该相信它？

因为可信度从来不自来"AI 不会错",而来自"错了能被发现、能被定位、能被修正、并且不会重复犯同一个错"。

人类分析师也会出错，企业依然信任他们——不是因为他们不出错，而是因为围绕他们建立了一整套机制：数据可以复核、结论可以质疑、错误可以追责、经验可以沉淀。AI 员工需要的是同一套机制，而不是更强的模型。

我把这套机制拆成五层。

第一层：把可能出错的地方压缩到最小

大多数 AI 分析出错，错的不是"判断",是"数字"。

如果把一张 CSV 和一句"分析一下"丢给大模型，它给出的每一个数字都是**生成的**而不是**计算的**——可能对，可能错，而且不可复现。这种情况下企业确实不该相信它。

所以我的架构做了一件事：**把数值计算从 LLM 手里彻底拿走**。

```
analysis_flow 分两类步骤：
metric 步骤 → pandas 确定性计算 → 结果 100% 正确且可复现
llm 步骤    → 只接收计算好的指标，负责解释和建议
              → Prompt 明确禁止："你不允许计算任何数字"
```


结果是：AI 可能"解读错",但绝不会"算错"。 而"解读"这件事本来就是主观的，人类专家之间也会有分歧——企业对它的容错度天然更高。

再加一道数字幻觉检测：扫描 LLM 输出的所有数字，只要出现计算结果里没有的数，直接判定幻觉并重试。

第二层：让每个结论都能被追溯

信任的前提是可验证，而不是"你说是就是"。

报告里的每个数字都携带溯源信息：

本场进入率 8.2% ▾ ← 点击展开

公式：进入直播间人数 / 曝光人数

取值：12,460 / 152,000

数据来源：uploaded_data.csv 第 2-15 行

计算时间：2026-08-10 14:32:07

质疑者可以在 30 秒内自己验算一遍。 这和人类分析师被质疑时"你把明细发我看看"是完全一样的动作。

一个不能被验算的结论，无论对错都不该被信任；一个可以被验算的结论，即使偶尔出错也依然可用。

第三层：主动承认不确定性

一个会说"我不知道"的系统，比一个永远给答案的系统更值得信任。

系统在这些情况下会主动标注：

情况	系统行为
必填数据缺失	拒绝执行，明确列出缺哪些字段、从哪导出
样本量不足	降级执行，报告中标注"因仅有 1 场数据，本次未做同比分析"
数据存在异常值	标注"检测到第 7 行转化率为 143%，疑似数据错误，已排除"
归因置信度低	输出多个可能原因并标注可能性，而非硬给一个结论

报告结尾固定有「数据说明与局限」章节。这个设计的价值在于：它把"AI 的能力边界"显式地交给了使用者，让人来做最后的判断。

第四层：错误可以被系统性修正

人类分析师犯错，你会跟他复盘，他下次不再犯。AI 员工也必须有这个闭环。

发现错误

↓

定位是哪一层出的问题（架构让这件事变得可能）

├─ 数据层 → 输入数据本身有问题 → 补充校验规则
├─ 计算层 → 指标公式口径错了 → 修正 metric_dictionary
├─ 逻辑层 → 分析流程不合理 → 修改 analysis_flow
└─ 表达层 → 模型解读有偏差 → 调整 agent_prompt

↓

把这个错误案例加入 Golden Set

↓

修正后跑全量回归测试，确认没有引入新问题

↓

发布新版本（旧版本保留，可回滚）

关键在于：这个错误从此被永久固化在评测集里，以后任何一次修改都会被它检验。这是人类分析师做不到的——AI 员工的错误只需要犯一次。

第五层：人在回路，责任明确

企业信任的从来不是"AI 本身",而是"AI + 使用它的流程"。

- Skill 发布前必须经过创建者（业务专家）逐项确认
- 每个 Skill 有明确的 Owner，出问题有人负责
- 高风险决策，AI 只提供分析和建议，**决策权始终在手里**
- 完整的执行日志：谁在什么时候用哪个版本的 Skill 分析了什么数据，全程可审计

更本质的一点

应该被质疑的，其实是"完全信任"这个前提本身。

企业不需要一个"永远正确所以可以盲信"的 AI，那种东西不存在。企业需要的是一个**能力边界清晰、错误可定位、结论可验证、迭代可度量**的工具。

就像企业信任 Excel——不是因为 Excel 不会算错（公式写错就会算错），而是因为你能点开单元格看到公式，能验算，能修正。

我们要做的，是让 AI 员工达到 Excel 那个级别的可信度：**透明、可验证、可追责**。

总结

可信度不是模型能力，是系统设计。

我们不承诺 AI 不出错，我们承诺：数字不会错、结论可验算、错误能定位、同样的错不会犯第二次、责任始终有人担。

这套机制建立起来之后，企业信任 AI 员工的方式，就和信任一个新入职的分析师是一样的——**先验证，再放权**。

Q6. Skill 和 Workflow、Agent、Prompt 三者有什么区别？

这四个词不在同一个维度上，所以"比较"之前要先分层。

- Prompt 是指令层——一次性的交流内容
- Workflow 是编排层——预先定义好的执行路径
- Agent 是执行层——具备工具和自主决策能力的运行时
- Skill 是知识层——被结构化封装的领域能力

总结：Prompt 是话术，Workflow 是流程，Agent 是执行者，Skill 是知识包。

四者对比

维度	Prompt	Workflow	Agent	Skill
本质	一段指令文本	预定义的执行图	有工具、能自主决策的运行时	结构化封装的领域能力资产
决策权	无	无（路径写死）	模型自主决定下一步	无（它是被调用的知识）
状态	无状态	有流程状态	有会话状态和记忆	有版本状态
确定性	低	高	低	可控（数值确定、解读灵活）
灵活性	高	低	高	中（框架固定，解读灵活）
可复用	靠复制粘贴	高（但改起来重）	中	高（平台统一分发）
可管理	几乎没有	有（流程引擎管）	弱	强（版本/权限/评测/审计）
典型载体	一段文字	DAG / 状态机	循环 + 工具调用	JSON Schema
谁来写	任何人	工程师	工程师	业务专家（通过自然语言）

两两辨析

Skill vs Prompt —— 资产 vs 消耗品

Prompt 是一次性消耗品：说完就没了，散落在个人聊天记录里，每个人写的都不一样。

Skill 是资产：有版本、有归属、有评测分、有输入输出契约、可被检索和授权。

注意：Skill 内部包含 Prompt (agent_prompt 字段)，但 Skill ≠ Prompt。就像"一本书里有文字，但书不等于文字"——书还有目录、章节结构、索引、版次。Skill 的那六个字段，就是围绕 Prompt 建立的结构。

Skill vs Workflow —— 知识 vs 路径

Workflow 是"先做 A 再做 B，如果 X 则做 C"——路径写死，确定但不灵活。遇到没预设的情况就卡住。

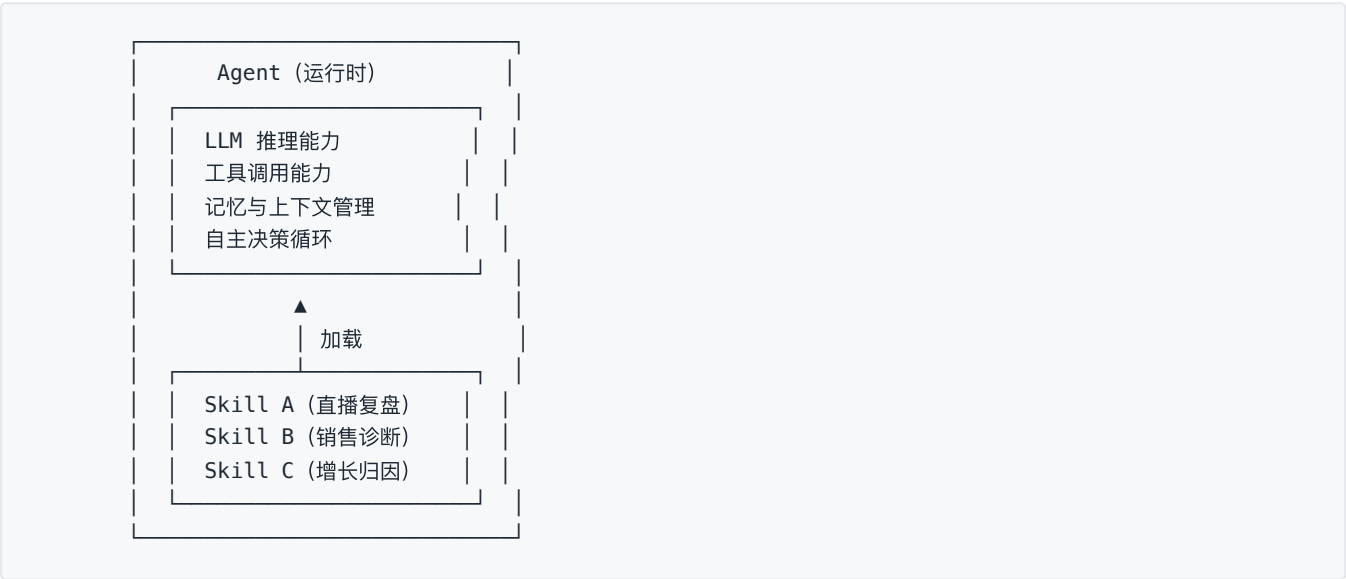
Skill 里也有流程 (analysis_flow)，但两者关键差别在于：

	Workflow	Skill 的 analysis_flow
每步做什么	固定的代码/规则	分两类：确定性计算 + LLM 推理
遇到意外情况	报错或走兜底分支	LLM 步骤可以理解上下文灵活应对
谁能修改	工程师改代码	业务专家改配置
目的	保证流程被执行	保证经验被复用

一句话：Workflow 关心"事情有没有按步骤做完",Skill 关心"判断有没有按专家的思路做对"。

Skill vs Agent —— 知识包 vs 运行时

这两个最容易混淆，但其实是最清晰的一组：Agent 是运行时，Skill 是喂给运行时的知识。



- 同一个 Agent 可以加载多个 Skill —— 一个"经营分析 AI 员工"可以同时具备复盘、诊断、归因三项能力
- 同一个 Skill 可以被不同 Agent 使用 —— 直播复盘 Skill 既可以被对话式 Agent 调用，也可以被定时任务调用
- Skill 让 Agent 从"通用助手"变成"岗位专家"

类比：Agent 是员工，Skill 是他受过的岗位培训。换个员工，培训还在；同一个员工，可以受多项培训。

总结

Prompt 是你交代的一句话，
Workflow 是你规定的一套流程，
Agent 是替你干活的那个人，
Skill 是这个人受过的岗位培训——它决定了他"懂不懂行"，而不是"能不能干"。
企业真正稀缺的，从来不是能干活的 Agent，而是懂行的 Skill。

Q7. 如果让你从零搭建这个平台，你会如何拆解 MVP？

我的拆解原则是：MVP 要验证的是"最不确定的那个假设",而不是"做出最多的功能"。

这个平台最不确定的假设是：

业务人员用自然语言描述的经验，能不能被自动转换成一个真正可执行、且结论可信的 Skill？

所以 MVP 的边界很清楚：凡是为验证这个假设服务的都做，其余全部砍掉。

第一步：识别真正的风险点

假设	风险等级	MVP 要不要验证
LLM 能生成结构合理的 Skill	⚠️ 中	✅ 必须
生成的 Skill 能真的执行出正确结果	🔴 高	✅ 核心
业务人员认可 AI 生成的分析框架	🔴 高	✅ 核心
用户愿意上传数据	🟡 低	用 CSV 上传即可，不做数据源对接
系统能支撑高并发	🟢 无	❌ 完全不做
Skill 能被多租户隔离管理	🟢 无	❌ 完全不做

结论：MVP 的重心在"生成质量"和"执行可信度",不在工程规模。

第二步：MVP 范围（做什么 / 不做什么）

模块	✅ MVP 做	❌ MVP 不做
Skill 生成	自然语言 → 七字段结构化生成 + 四道校验闸	多轮对话澄清、语音输入、批量生成
Skill 配置	可视化展示 + 关键字段可编辑	拖拽式流程编辑器
Skill 存储	SQLite + 版本号	数据库集群、多租户、权限体系
数据接入	CSV 上传 + 字段映射	数据库直连、API 对接、实时数仓
Skill 执行	metric 确定性计算 + llm 归因	定时调度、批量执行、异步队列
结果输出	Markdown 报告 + 溯源展示	PDF 导出、飞书/钉钉推送、邮件订阅
质量保障	Schema 校验 + 幻觉检测 + 1 组 Golden Set	完整评测平台、专家评审 workflow
预置内容	3 个预置 Skill + 配套示例数据	Skill 市场、行业模板库
用户体系	无（单用户）	登录、权限、组织架构

砍掉用户体系这件事我想特别说明： 它看起来是"企业级产品必备",但它对验证核心假设毫无帮助，而实现成本不低。这种功能应该在核心假设被验证之后再做。

第三步：实施顺序（我会按这个顺序写代码）

关键原则：先打通最短路径，再逐层加固。

【D1-2】定义 SkillSpec Schema ← 起点，全系统的中心

- Pydantic 模型
- 手工写 1 个完整的 Skill 样例（不用 AI 生成）
- 目的：先把"目标形态"定死，后面所有模块都朝它对齐

【D3-4】打通执行链路（先做执行，后做生成）★

- CSV 加载 → input_schema 校验
- metric 计算引擎 (ast 安全求值)
- llm 归因步骤
- 报告渲染
- 验证：手工写的 Skill + 示例数据 → 能跑出正确报告

【D5-6】打通生成链路

- Prompt 设计 + few-shot
- LLM structured output
- 四道校验闸
- 验证：一句话 → 生成的 Skill 能被上一步的执行引擎跑通

【D7-8】前端界面

- 生成页 / 配置展示页 / 执行页 / 报告页
- Tailwind 保证视觉质量

【D9】质量层

- 数字幻觉检测
- 溯源信息展示
- 1 组 Golden Set + 评分

【D10】收尾

- 补齐 3 个预置 Skill
- README + 部署 + 演示视频

为什么先做执行、后做生成？

这是个反直觉但很重要的决定。因为：

1. 执行链路定义了"什么样的 Skill 才是合格的"。先把执行引擎写出来，"合格 Skill"的标准就变成了客观的、可运行验证的，而不是主观感觉
2. 如果先做生成，会陷入"生成了一堆看起来很美但根本跑不通的 Skill"的陷阱——这恰恰是这类产品最常见的失败方式
3. 执行链路一旦跑通，生成链路的验收标准就一句话：**生成的 Skill 能不能被执行引擎跑通**

第四步：MVP 的验收标准

不是"功能做完了",而是能回答这三个问题：

#	验收问题	判定方式
1	一句话生成的 Skill， 能不能被真实执行？	随机 10 次生成，跑通率 ≥ 80%
2	执行出的报告， 数字对不对？	Golden Set 数据准确性 = 100%
3	业务专家看了生成的分析框架， 认不认可？	找 2 位有经验的运营看，能不能说出"这个思路是对的"

第 3 条是最重要也最容易被忽略的。前两条是工程问题，做得出来；第 3 条才是产品问题——如果专家看了说"不对，我不是这么看的",那前面做得再工整都没有意义。

第五步：MVP 之后的演进

阶段	目标	关键能力	触发条件
V1 质量可信	让企业敢用	完整评测集、回归测试、版本管理、审批流、权限体系	MVP 验收通过
V2 数据打通	降低使用门槛	数据源直连、定时执行、异常自动预警	有 ≥ 3 个真实企业在用
V3 资产运营	规模化复制	Skill 市场、行业模板库、跨 Skill 编排	单企业 Skill 数 ≥ 30
V4 组织协同	成为基础设施	多 Agent 协作、AI 员工绩效看板、OA/IM 集成	——

每个阶段都有明确的触发条件，不到条件不启动。这是为了避免最常见的失败模式：核心价值还没被验证，就急着堆功能。

总结

MVP 不是"功能最少的版本",是"验证核心假设所需的最小完整闭环"。

这个平台的核心假设是"自然语言能变成可执行且可信的 Skill",所以我的 MVP 保留了完整的**生成 → 校验 → 展示 → 执行 → 输出**闭环，但砍掉了用户体系、数据对接、调度等一切与验证无关的东西。

实施上我会反直觉地先做**执行链路**再做**生成链路**——因为只有执行引擎存在，"什么是合格的 Skill"才有客观标准，生成才不会跑偏。